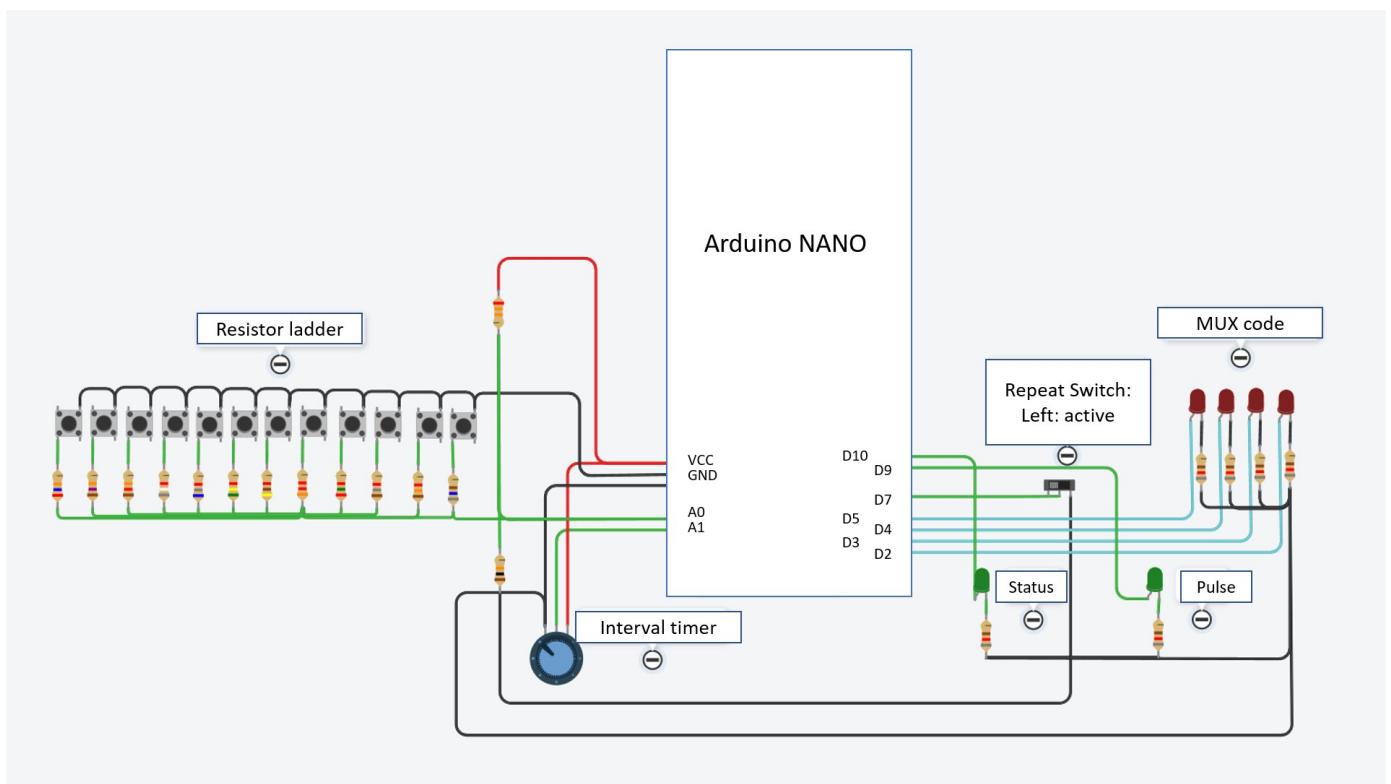


Automating the Yaesu FH-2 with an Arduino NANO

As Radio Scouts we are involved and organize several HF radio activities. Depending on conditions and the length of the event it could very handy to use a voice keyer. Our inventory consists mainly of Yaesu equipment so the voice keyer choice is limited to a FH-2 remote device. Unfortunately, repetition of the recall of a memory slot depends on the sole finger action of the operators in charge, sometimes supported by a scout who sits aside the operator. It was during poor conditions, hence repetative finger operations on the FH-2, that the idea was born to make a **FH-2 Repeater** which repeats memory slot recalls with predefined (but adjustable) interval times.



The FH-2 Repeater described in this article is a small, Arduino-based controller that plugs into the Yaesu FH-2 Morse keyer connector of your Yaesu transceiver and sends your pre-programmed messages automatically, at whatever interval you choose. It is not a commercial product. It is a homebrew project, designed from scratch using only through-hole components, and it invites you to build it yourself, improve upon it, and make it your own.



Testing the design, ideas and software the [TinkerCad online simulator](#) is an ideal tool. Together with a helpfull AI machine the design was fine-tuned and developed before it was taken to the breadboard components.

What Does It Do?

The FH-2 Repeater takes over the repetitive task of triggering the FH-2's memory keyer buttons at regular intervals. In practical terms, you press a button on the FH-2 (or on the controller), and from that moment onward the controller automatically resends that same message every 10 to 60 seconds — the exact interval is set by turning a front-panel potentiometer. You carry on with logging, antenna switching, or enjoy a well-deserved cup of coffee.

A front-panel toggle switch turns the repeat function on and off entirely. Two LEDs give you at-a-glance confirmation: one flashes each time a transmission is triggered, and the other glows steadily to tell you that an automatic cycle is running. If you need to stop immediately — a rare opening on another band, an urgent call on the radio — pressing any button other than the active one cancels the cycle instantly.

All activity is also reported in real time over the serial port at 9600 baud, complete with timestamps, so you can connect a laptop, open the serial monitor, and have a detailed log of every event during your operating session.

```
[00012] Button 1 detected (voltage: 0.42V)
[00012] Interval cycle started, interval: 30s
[00042] Pulse sent: channel 1
[00072] Pulse sent: channel 1
[00083] Interval changed: 30s -> 45s
[00128] Pulse sent: channel 1
[00173] Cycle stopped by operator
```

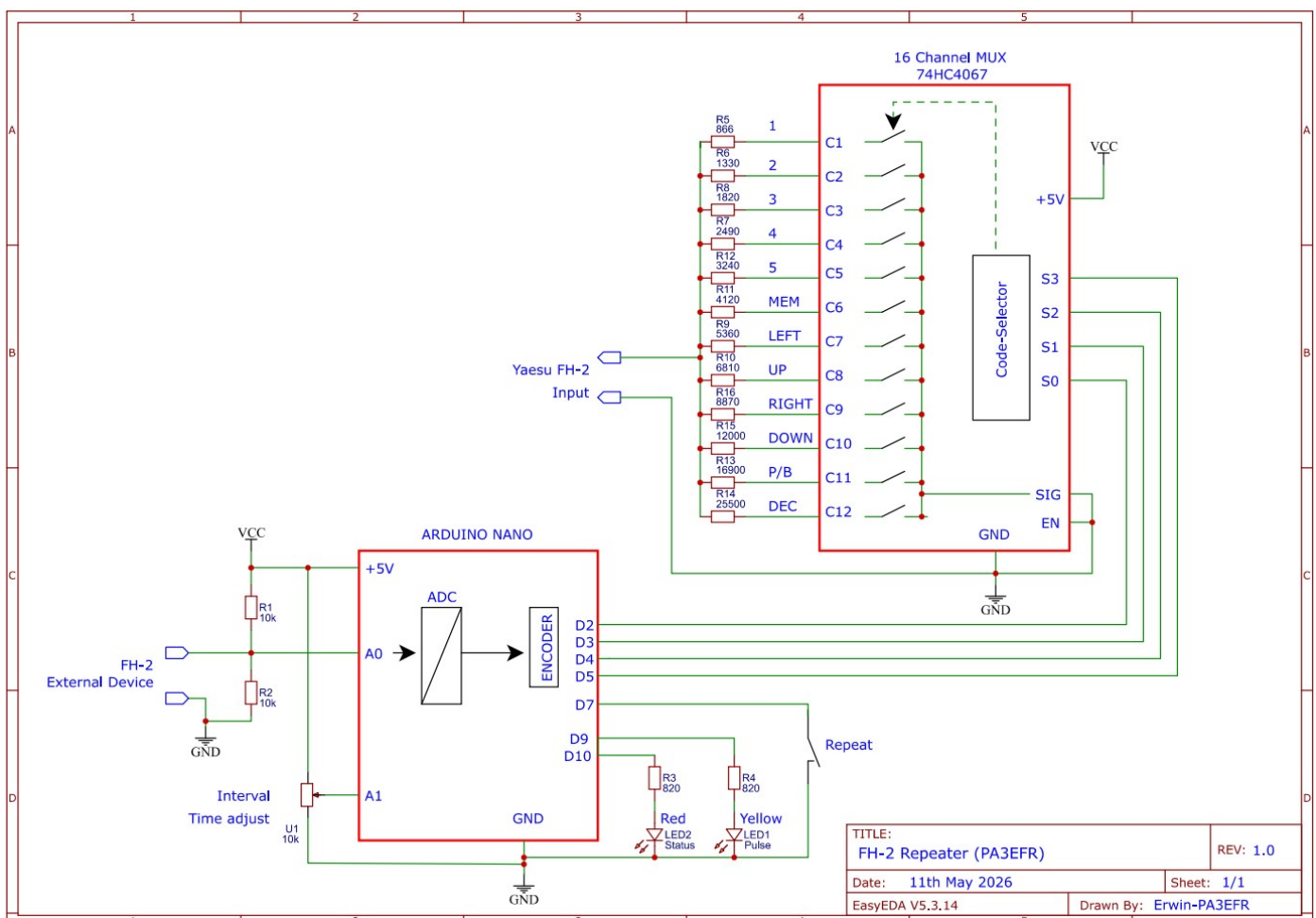
How Does It Work?

Reading the FH-2 Button Matrix

The FH-2 exposes its button matrix through a resistor network, and each button produces a distinct voltage on the output line. The controller measures this voltage with the Arduino's built-in analogue-to-digital converter on pin A0. Each of the FH-2 buttons produces a characteristic voltage between about 0.38 V and 2.53 V, and the software compares each reading against a lookup table, accepting a match if it falls within a ± 0.15 V tolerance window. A best-match algorithm handles the edge cases, making detection robust against minor power supply variations or temperature drift.

The Multiplexer — The Key to Pressing Buttons Electronically

To send a button press to the FH-2, the controller needs to briefly close the same electrical contact that a physical button would close. This is done through a 74HC4067 sixteen-channel analogue multiplexer — a



cheap, widely available CMOS chip that costs only a few cents. Four address lines from the Arduino select which of the sixteen channels is connected, and an active-low enable line controls when the output is active.

When a transmission is due, the Arduino selects the correct channel, enables the multiplexer for 250 milliseconds, then disables it and resets to channel zero. A 2-second timeout in the software ensures the multiplexer always returns to a safe state, even if something unexpected interrupts the sequence.

The Interval State Machine

The heart of the software is a simple three-state machine: idle, active-cycling, and stopped. Pressing buttons 1 through 5 while the repeat switch is on moves the system into the active-cycling state. From that point, the software checks the elapsed time against the potentiometer reading on every pass through the main loop — running at approximately 100 Hz — and triggers a new pulse when the interval expires. Turning the potentiometer takes effect immediately, without interrupting the current cycle, which is very useful for real-time adjustments during a contest.

Debounce and Reliability

Mechanical switches bounce — they make and break contact several times in the first millisecond after being pressed. The controller deals with this by requiring two identical consecutive readings separated by 50 milliseconds before accepting a button press as genuine. This simple debounce algorithm is robust in practice and adds no perceptible delay for the operator.

Circuit and Components

The circuit is deliberately simple. There are no surface-mount components, no exotic parts, and nothing that cannot be sourced from any of the usual suppliers. The complete bill of materials is short:

- Arduino NANO (ATmega328P) — the processing core
- 74HC4067 16-channel analogue multiplexer — the button emulator
- 10 kΩ linear potentiometer — interval adjustment
- SPDT toggle switch — repeat on/off
- Two LEDs with appropriate series resistors (pulse and status)
- 3.5 mm stereo jack socket — FH-2 connection
- Assorted resistors and decoupling capacitors
- Stripboard or a custom PCB

The Arduino is powered from USB during development and can be powered from an external 5 V regulator in a permanent installation. Total current consumption is modest — around 100 mA at most.

Function	Pin	Description
ADC Input	A0 (Pin 14)	FH-2 button voltage measurement
Interval Control	A1 (Pin 15)	Potentiometer wiper for interval adjustment
MUX S0–S3	D2–D5 (Pins 2–5)	4-bit multiplexer channel selection
MUX Enable	D6 (Pin 6)	Active-low enable for 74HC4067
Repeat Switch	D7 (Pin 7)	Pull-up input for repeat on/off toggle
Pulse LED	D9 (Pin 9)	Visual pulse confirmation indicator
Status LED	D10 (Pin 10)	Interval cycle active indicator

The Software

The firmware is written in standard Arduino C++ and follows a straightforward modular structure. There are separate functions for button detection, multiplexer control, pulse generation, and serial reporting. The main loop uses a cooperative multitasking approach, cycling through all inputs and outputs at roughly 100 Hz. This keeps the code easy to read, easy to modify, and easy to extend.



Build It Yourself — You Can Do This

This project is accessible to any amateur who has soldered a few components and loaded a sketch onto an Arduino. If you have never done either of those things, this is an excellent first project: the circuit is forgiving, the components are inexpensive, and the Arduino ecosystem has outstanding documentation and a large, helpful community.

The through-hole construction is intentional. Every component can be held in place with two fingers while you solder it. Every joint can be inspected and reflowed if needed. There are no heat-sensitive plastic packages to worry about, no tiny pads to bridge accidentally, and no need for a microscope or hot-air re-work station.

A Note for Those Who No Longer Solder

Not every amateur is in a position to build a project like this. Perhaps your eyesight makes fine soldering difficult, or your hands are no longer steady, or you simply do not have a workshop any more. That is entirely understood, and it is not a failure.

The intention of publishing this design openly is not to exclude, but to share. If the project interests you, perhaps a club member or a younger amateur in your circle would be willing to build it for you — a joint effort that passes on skills as well as producing a useful piece of equipment. The design is simple enough that it makes an excellent mentoring project.

For Jamboree On The Air this could well be a good solder project in the summer months for the scouts to take on and use during the JOTA in October. You will not regret the effort.

Alternatively, the software and circuit description here may be enough to inspire someone you know to adapt the principle to a different controller or a different keyer altogether. Good ideas have a way of multiplying.

Conclusion

The FH-2 Repeater is a practical solution to a real problem, built from components that cost less than a contest entry fee and assembled in an evening. It is not the last word in automation, and it was never intended to be. It is a starting point: a working, [documented design](#) that you can build, understand, modify, and improve.

On behalf of the Radio Scouting Fellowship of PA3EFR/J, 73 and good luck with the build.

